RepoGuard

A static scanner that detects malicious GitHub repositories
(fake job-interview and coding-test scams) before you clone or run them

6-week plan for two people working in parallel

1. Project goal

Many developers get a "take-home assignment" repository that runs malware the moment they install dependencies or open the folder in their editor. Existing tools (ghsafe, Flagrix, sketchy, GuardDog, Semgrep) cover parts of this problem. RepoGuard focuses on one sharp niche: **fake interview / coding-test repositories**, with extra checks for places general scanners overlook, repo and author reputation signals, plain-language explanations, and a measured false-positive rate.

Definition of done (end of week 6): a pip-installable CLI that scans a GitHub URL or local folder *without executing anything*, outputs a risk score with explained findings (terminal, JSON, HTML), ships with tests, a GitHub Action, and a public evaluation report.

2. Roles: who owns what

The project is split into two tracks that meet at one shared interface. This lets both of you code every week without blocking each other.

	Partner A - Detection Engine	Partner B - Repo Intelligence and UX
Owns	Rules, file analysis, obfuscation and install-script detection, rule format (YAML), unit tests with fake samples.	GitHub API fetching, repo/author reputation signals, risk scoring, reports (terminal, JSON, HTML), CLI, packaging, CI.
Skills gained	Static analysis, regex and AST parsing, how real malware hides, testing, rule tuning.	REST APIs, rate limits, data modeling, CLI design, report generation, Python packaging, GitHub Actions.
Main folders	`rules/ scanner/ tests/`	`fetcher/ reputation/ report/ cli.py`

Tip: swap tasks for part of Week 5 so each of you understands the other half. You can also decide who is A and who is B by flipping a coin; both tracks teach valuable skills.

3. The shared contract (agree on this in Week 1)

These three pieces are the only places your code touches. Write them together on day one, then you can work independently.

```
@dataclass
class Finding:               # produced by A's scanner
    rule_id: str             # e.g. "NPM-POSTINSTALL-REMOTE"
    severity: str            # "low" | "medium" | "high"
    file: str
    line: int
    snippet: str
    explanation: str         # plain-language "why this is dangerous"
    advice: str              # "what to do"

@dataclass
class RepoMeta:              # produced by B's fetcher / reputation module
    account_age_days: int
    commit_count: int
    stars: int
    is_fork: bool
    flags: list[str]         # e.g. ["single-commit", "new-account"]

# A:  scan_files(files: Iterable[tuple[str, str]]) -> list[Finding]
# B:  fetch_repo(url) -> (files: Iterable[tuple[str, str]], meta: RepoMeta)
# B:  score(findings, meta) -> Risk(level="low|medium|high", points=int)
```

4. How to work together

- **One shared GitHub repo.** Never commit directly to `main`. Each task = a branch (`feat/obfuscation-rules`) and a pull request.
- **Cross-review every PR.** A reviews B's code and B reviews A's. This is the fastest way to learn and it looks great to employers.
- **Two short syncs per week (20-30 min):** Monday to plan, Thursday or Friday to demo and merge. Keep a shared notes file (`docs/devlog.md`).
- **Use GitHub Issues and a Project board** (To do / Doing / Review / Done). Each weekly task below becomes an issue.
- **Small commits, clear messages,** and tests with every feature. Merge only when tests pass.
- **If one of you falls behind,** the other writes the next task as a stub against the contract rather than waiting.

5. Week-by-week plan

Legend: blue = Partner A (Detection Engine), orange = Partner B (Repo Intelligence and UX), grey = done together.

Week 1 - Research, setup, and the contract

Partner A - Detection Engine	Partner B - Repo Intelligence and UX
• Run GuardDog, sketchy, and ghsafe on 3-4 repos; note what each catches and misses. • Read about fake-interview malware campaigns; list 10 techniques attackers use. • Draft the rule list (which patterns to detect first).	• Create the repo, folder structure, README skeleton, license, and issue board. • Set up Python environment, linting, pytest, and a pre-commit hook. • Test the GitHub API: fetch a repo's file tree and a file's contents with and without a token.
Together: Write the shared contract (Finding, RepoMeta) together. Milestone: contract merged and both tracks can start.	

Week 2 - First working version (v0.1)

Partner A - Detection Engine	Partner B - Repo Intelligence and UX
• File walker over a local folder. • 5-6 rules: eval/exec, base64 blobs, very long lines, curl-pipe-shell, hidden Unicode characters. • Unit tests using your own harmless fake samples.	• Basic CLI: `repoguard scan ./folder` that calls A's scanner. • Terminal report with colors and severity levels. • Fetcher v1: download a repo's files through the GitHub API into memory.
Together: Integrate. Milestone: scan a local folder and print findings end to end.	

Week 3 - Repo-specific checks and the rule format

Partner A - Detection Engine	Partner B - Repo Intelligence and UX
• Parse `package.json` scripts (preinstall/postinstall), `setup.py`, Makefile. • Check `.vscode/tasks.json` (auto-run on folder open) and `.devcontainer`. • Move rules into YAML files so new rules need no code changes.	• Fetcher v2: scan a GitHub URL without cloning, with file-size limits and a cap on files fetched. • Handle rate limits, errors, private repos, and missing files gracefully. • Start RepoMeta: account age, commit count, stars, fork status.
Together: Milestone: `repoguard scan https://github.com/...` works on a real public repo.	

Week 4 - Reputation signals and risk scoring

Partner A - Detection Engine	Partner B - Repo Intelligence and UX
• GitHub Actions checks (e.g. `pull_request_target`), git hooks, Docker socket access. • Typosquat check: compare dependency names to a list of popular packages. • Improve rules to reduce false positives.	• Reputation flags: new account, single forged-date commit, star spikes, copied fork. • Risk score (low/medium/high) combining findings and metadata; document how points are assigned. • Plain-language explanation and advice for each finding.
Together: Milestone: every scan ends with a risk level and a "what to do" summary.	

Week 5 - Usability, reports, and packaging

Partner A - Detection Engine	Partner B - Repo Intelligence and UX
• Suppression/allowlist file so users can silence known-safe findings. • Add 10+ new tests; reach solid coverage of all rules. • (Swap half-day) Review and extend one of B's modules.	• JSON and HTML report output with line previews. • `safeclone <url>` wrapper: scan first, clone only if the user confirms. • Package for `pip install`; write installation docs. • (Swap half-day) Review and extend one of A's modules.
Together: Milestone: a friend can install and use the tool from the README alone.	

Week 6 - Evaluation and publishing

Partner A - Detection Engine	Partner B - Repo Intelligence and UX
• Build a set of 15-20 handmade fake-malicious fixtures; measure detection rate. • Tune rules based on results; document known limitations (what it cannot catch).	• Run the scanner on 50-100 popular, trusted repos; count and categorize false positives. • Create a GitHub Action so others can use it in CI. • Record a short demo GIF for the README.
Together: Write the README and a joint blog/LinkedIn post: problem, approach, results (detection rate, false positives), lessons, and next steps. Milestone: v1.0 release tag.	

6. Suggested repository structure

```
repoguard/
  README.md
  pyproject.toml
  docs/
    devlog.md              # weekly notes (both of you)
    rules.md               # what each rule detects and why
  src/repoguard/
    cli.py                 # B
    models.py              # shared contract (both)
    scanner/               # A: file walker, rule engine
    rules/                 # A: YAML rule files
    fetcher/               # B: GitHub API loader
    reputation/            # B: account/repo signals
    scoring.py             # B
    report/                # B: terminal, JSON, HTML
  tests/
    fixtures/              # A: harmless fake-malicious samples
    test_rules.py          # A
    test_fetcher.py        # B
  .github/workflows/       # B: tests + the GitHub Action
```

7. Safety and legal rules (non-negotiable)

- **The scanner must be static:** it reads text and never executes, imports, or installs anything from a scanned repo.
- **Never run suspicious code on your own machine.** If you ever need to study a real sample, use a disposable VM with no network and no personal data.
- **Write your own fixtures.** Test files should imitate patterns (for example a harmless script that contains an `eval` and a base64 string), not contain working malware.
- **Respect GitHub's terms and rate limits.** Use a personal token stored in an environment variable and never commit it to the repo.

- **Be careful when naming repos publicly.** In your evaluation report, describe false-positive categories rather than shaming specific maintainers.
- **Label results honestly:** a risk score is a warning, not proof. Document what the tool cannot detect.

8. After week 6: upgrade ideas

- Browser extension that shows a risk badge on GitHub repo pages.
- npm and PyPI package scanning mode (the same rules apply to packages).
- Entropy-based secret detection (links to your secrets-scanner idea).
- Optional AI-written summary of flagged files, with the rule-based result always shown first.
- A public list of rule ideas so others can contribute, and a small community of users.

9. Checklist for v1.0

[ ]	Scans a GitHub URL and a local folder without executing code
[ ]	At least 15 rules, including install scripts, editor auto-run files, and obfuscation
[ ]	Reputation signals included in the risk score
[ ]	Plain-language explanation and advice for every finding
[ ]	Terminal, JSON, and HTML output
[ ]	Tests passing in CI; documented false-positive rate
[ ]	README with demo, limitations, and install steps
[ ]	Public write-up published (blog or LinkedIn) by both of you